INTELIGÊNCIA ARTIFICIAL · AULA 6.8

Naive Bayes do zero

Como um classificador toma decisões com probabilidades calculadas a partir dos dados

A IDEIA CENTRAL DA AULA

1. **Mensagem**
   “Parabéns! Você ganhou uma promoção!”
2. **Contagens**
   quantas vezes cada palavra aparece
3. **Pontuação por classe**
   spam e não spam
4. **Decisão**
   a classe com a maior pontuação

2 horas Python + NumPy Exercício: filtro de spam

Objetivos e roteiro da aula

PROBABILIDADE

- Probabilidade e probabilidade condicional
- Teorema de Bayes
- Priori, verossimilhança e posteriori

O MODELO

- O significado de “Naive”
- Independência condicional à classe
- Gaussian e Multinomial

TEXTO

- Textos como contagens de palavras
- Suavização de Laplace
- Logaritmos contra o underflow

IMPLEMENTAÇÃO

- Classificador em NumPy
- Filtro de spam do zero
- Acurácia e matriz de confusão

ROTEIRO · MINUTOS	
0–10	Objetivos e recapitulação
10–30	Probabilidade condicional e Bayes
30–45	Hipótese do Naive Bayes
45–60	Gaussian e Multinomial
60–80	Laplace, logaritmos, cálculo manual
80–110	Exercício guiado em NumPy
110–120	Quiz, erros comuns, desafio

O que já sabemos sobre classificação

SUPERVISIONADO

Exemplos com características X e rótulos y. Binário: spam = 1, não spam = 0.

REGRESSÃO LOGÍSTICA

Estima a probabilidade da classe 1 com a sigmoide.

$$P(y = 1 \mid x)$$

lê-se “probabilidade de y = 1 dado x”; a barra é detalhada no slide 7

KNN · AULA 6.7

Classe da maioria entre os k vizinhos mais próximos, por distância.

MÉTRICAS

Acurácia, matriz de confusão, precisão e recall, sempre no teste.

AULA 6.7 · KNN

Quem está perto desta mensagem?

HOJE · NAIVE BAYES

Com que frequência estas palavras aparecem em cada classe?

Um problema real: esta mensagem é spam?

> “Parabéns! Você ganhou uma promoção!”

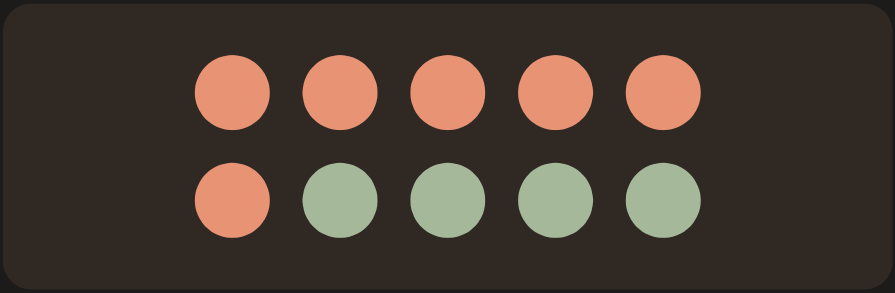
Ideia: comparar com que frequência cada palavra aparece em mensagens de spam e em mensagens normais já rotuladas.

CONTAGENS NO TREINO DO EXERCÍCIO GUIADO · 12 SPAM, 12 NORMAIS

PALAVRA	EM SPAM	EM NORMAIS
parabéns	2	0
você	2	1
ganhou	1	0
uma	0	0
promoção	4	0

Como transformar contagens em uma decisão?

Probabilidade: contar casos favoráveis



$$P(\text{Vermelha}) = \frac{6}{10} = 0{,}6 = 60\%$$

ESPAÇO AMOSTRAL · Ω

Todos os resultados possíveis: as 10 bolas.

EVENTO

O resultado que nos interessa: “sair vermelha”, 6 casos.

PROBABILIDADE

Casos favoráveis ÷ total de casos.

ESCALA

Sempre entre 0 (impossível) e 1 (certo). 0,6 = 60%.

Probabilidade teórica e estimativa empírica

TEÓRICA

Conhecemos o processo: a caixa tem 6 vermelhas em 10.

$$P(\text{Vermelha}) = \frac{6}{10} = 0{,}6$$

EMPÍRICA · ESTIMADA COM DADOS

Não conhecemos a caixa. Exemplo: 50 retiradas com reposição, 28 vermelhas.

$$\hat{P}(\text{Vermelha}) = \frac{28}{50} = 0{,}56$$

$$\hat{P}(\text{Spam}) = 40/100 = 0{,}4$$

No Naive Bayes, toda probabilidade é uma frequência relativa medida no conjunto de treino. O chapéu em $\hat{P}$ indica estimativa.

Probabilidade condicional: o símbolo “ | ”

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

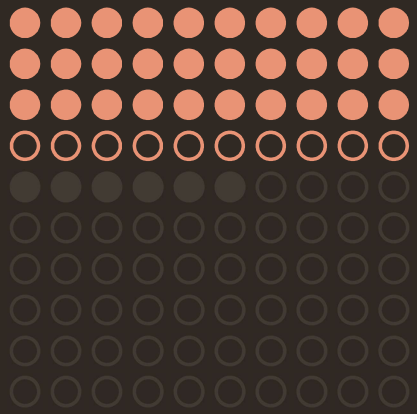
$\mid$	“dado que”: a condição já conhecida
A	o evento que queremos medir
B	a informação observada; P(B) > 0
$\cap$	“e”: A e B acontecem juntos

100 MENSAGENS DE EXEMPLO

	COM “GRÁTIS”	SEM “GRÁTIS”	TOTAL
Spam	30	10	40
Normal	6	54	60
Total	36	64	100

$$P(\text{Grátis} \mid \text{Spam}) = \frac{30}{40} = 0{,}75$$

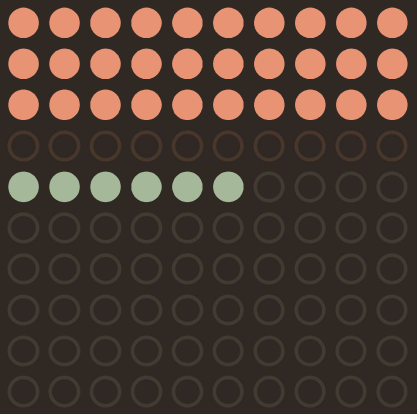
As mesmas 100 mensagens, duas perguntas diferentes



CONDIÇÃO: É SPAM · 40

$$P(\text{Grátis} \mid \text{Spam}) = \frac{30}{40} = 0{,}75$$

Contamos só entre os 40 spams.



CONDIÇÃO: TEM “GRÁTIS” · 36

$$P(\text{Spam} \mid \text{Grátis}) = \frac{30}{36} \approx 0{,}8333$$

Contamos só entre as 36 mensagens com “grátis”.

● spam com “grátis” · 30 ○ spam sem · 10 ● normal com “grátis” · 6 ○ normal sem · 54

Mesmo numerador (30), denominadores diferentes: a condição define o universo.

Quiz rápido: probabilidade condicional

Q1 O que significa P(A | B)? Leia em voz alta.

Q2 P(A | B) é igual a P(B | A)? Dê um exemplo.

Q3 Calcule P(Normal | Grátis).

Q4 Calcule P(Grátis | Normal). Por que é diferente de Q3?

USE A TABELA

	GRÁTIS	SEM	TOTAL
Spam	30	10	40
Normal	6	54	60
Total	36	64	100

Duplas · 90 segundos. Respostas comentadas nas notas.

De onde vem o Teorema de Bayes

Duas maneiras de calcular a mesma interseção.

1	$P(A \mid B) \cdot P(B) = P(A \cap B)$	Definição de condicional, multiplicada por P(B).
2	$P(B \mid A) \cdot P(A) = P(A \cap B)$	A mesma definição com a condição invertida.
3	$P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$	As duas linhas calculam a mesma interseção.
4	$P(A \mid B) = \dfrac{P(B \mid A) \cdot P(A)}{P(B)}$	Dividimos por P(B) > 0. Teorema de Bayes.

CONFERINDO Spam com “grátis”: $0{,}75 \times 40 = 30$ e $0{,}8333 \times 36 = 30$

Os quatro termos do Teorema de Bayes

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

PRIORI

$P(\text{Spam})$

Crença antes de ler a mensagem.

0,4

VEROSSIMILHANÇA

$P(\text{Grátis} \mid \text{Spam})$

Chance da evidência se a classe for spam.

0,75

EVIDÊNCIA

$P(\text{Grátis})$

Chance da palavra em qualquer mensagem.

0,36

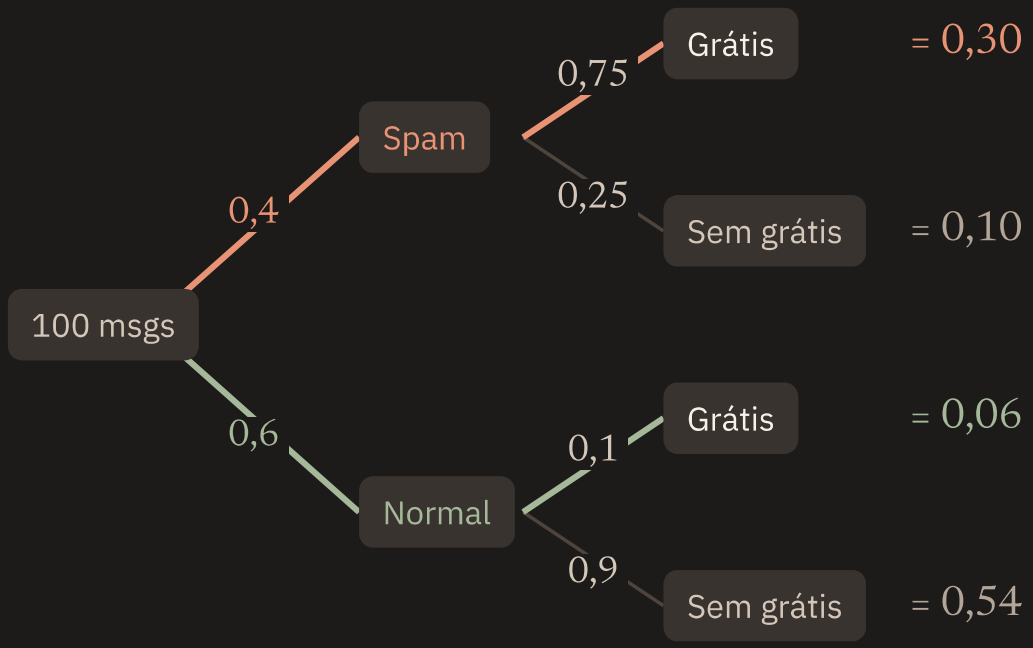
POSTERIORI

$P(\text{Spam} \mid \text{Grátis})$

Crença depois de ver a evidência.

≈ 0,8333

Bayes com números: a árvore de probabilidades



1 · EVIDÊNCIA: SOME OS CAMINHOS COM "GRÁTIS"

$$P(\text{Grátis}) = 0{,}30 + 0{,}06 = 0{,}36$$

2 · POSTERIORI

$$P(\text{Spam} \mid \text{Grátis}) = \frac{0{,}75 \times 0{,}4}{0{,}36}$$

$$= \frac{0{,}30}{0{,}36} \approx 0{,}8333$$



O que significa o nome Naive Bayes

BAYES

Usa o Teorema de Bayes

Inverte o que medimos no treino, P(palavras | classe), no que queremos na predição, P(classe | palavras).

NAIVE · INGÊNUO

Faz uma hipótese simplificadora

Dentro de cada classe, supõe que as características não se influenciam: independência condicional à classe.

TREINO · APENAS CONTA

Prioris $P(C)$ e verossimilhanças $P(x_i \mid C)$ para cada classe e característica.

PREDIÇÃO · COMPARA

Uma pontuação por classe. A classe prevista é a de maior pontuação.

A hipótese: independência condicional à classe

O QUE O NAIVE BAYES SUPÕE

$$P(\text{grátis, promoção} \mid \text{Spam})$$
$$= P(\text{grátis} \mid \text{Spam}) \times P(\text{promoção} \mid \text{Spam})$$

Ilustrativo: 0,75 × 0,5 = 0,375

A vírgula significa “e”: as duas palavras presentes na mesma mensagem.

O QUE ELE NÃO SUPÕE

~~$P(\text{grátis, promoção}) = P(\text{grátis}) \times P(\text{promoção})$~~

Sem olhar a classe, “grátis” e “promoção” andam juntas: ambas são típicas de spam. A hipótese vale só dentro de cada classe, e mesmo assim é uma aproximação.

FORMA GERAL $P(x_1, \ldots, x_n \mid C) = \prod_{i=1\ldots n} P(x_i \mid C)$ ∏ multiplica todos os termos, como Σ soma.

Por que uma hipótese ingênua funciona bem

1 · PARÂMETROS POR CLASSE, N PALAVRAS

Sem a hipótese · 2^n

1.048.576

para n = 20

302 dígitos

para n = 1000

Com a hipótese · n

20

para n = 20

1000

para n = 1000

2 · A DECISÃO SÓ USA A ORDEM

Basta a classe certa ter a maior pontuação, mesmo que os valores estejam distorcidos.

3 · TREINAR É CONTAR

Uma passagem pelos dados. Funciona com poucos exemplos e vocabulários grandes.

LIMITAÇÃO

Evidências correlacionadas são contadas em dobro: as probabilidades ficam extremas.

Como o Naive Bayes classifica uma mensagem

1 **Mensagem** — “grátis promoção”

→

2 **Características** — contagens de palavras

→

3 **Pontuação por classe** — priori × verossimilhanças

→

4 **Comparação** — spam × normal

→

5 **Classe prevista** — a maior pontuação

BAYES PARA A CLASSE C

$$P(C \mid x) = \frac{P(x \mid C) \cdot P(C)}{P(x)}$$

$P(x)$ é igual para todas as classes: não muda a ordem.

$x = (x_1, \ldots, x_n)$: as características da mensagem, aqui as contagens de palavras.

PONTUAÇÃO USADA PARA COMPARAR

$$P(C \mid x) \propto P(C) \prod_i P(x_i \mid C)$$

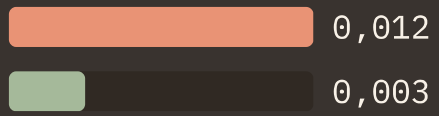
$\propto$ “proporcional a”: mesma ordem, sem somar 1.

Escolhendo a classe: argmax

$$\hat{y} = \text{argmax}_c \, P(C) \prod_i P(x_i \mid C)$$

$\hat{y}$ (“y chapéu”) = classe prevista · argmax devolve **qual** classe tem a maior pontuação

PONTUAÇÕES · ILUSTRATIVAS



MAX × ARGMAX

`max` → 0,012

`argmax` → Spam

`np.argmax([0.003, 0.012])` → 1

NORMALIZAR · OPCIONAL

$$\frac{0{,}012}{0{,}015} = 0{,}8 \qquad \frac{0{,}003}{0{,}015} = 0{,}2$$

A classe escolhida não muda.

Quiz rápido: o modelo Naive Bayes

Q1 O que é probabilidade a priori?

Q2 O que representa a verossimilhança?

Q3 O que é probabilidade a posteriori?

Q4 Por que o algoritmo se chama Naive Bayes?

Q5 O que significa independência condicional?

Q6 Como o algoritmo escolhe a classe?

Gaussian Naive Bayes: características contínuas

Altura, peso, temperatura, comprimento: valores reais, não contagens.



CLASSE A · COMPRIMENTOS 4, 4, 6, 6

μ · média

$$(4 + 4 + 6 + 6) \div 4 = 5$$

σ² · variância

$$(1 + 1 + 1 + 1) \div 4 = 1$$

σ · desvio-padrão

$$\sqrt{1} = 1$$

Uma média e uma variância por classe e por característica.

A densidade normal, símbolo por símbolo

$$f(x \mid C) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(- \frac{(x - \mu)^2}{2\sigma^2} \right)$$

$f(x \mid C)$	densidade · de x na classe C
x	valor observado · ex.: comprimento 6
C	classe · A ou B
μ	média · da classe, no treino
σ^2	variância · espalhamento
σ	desvio-padrão · $\sqrt{\sigma^2}$
π	pi · ≈ 3,1416
e, exp	Euler · e ≈ 2,7183; exp(z) = e^z

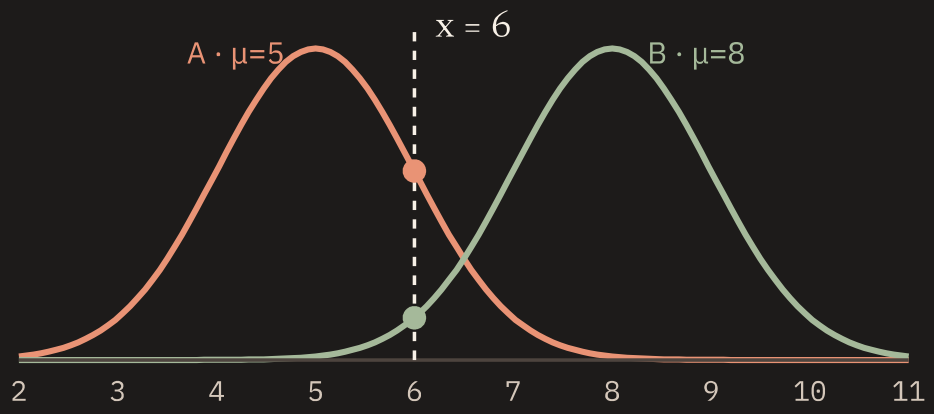
DENSIDADE ≠ PROBABILIDADE

Em dados contínuos, $P(X = 6 \text{ exato}) = 0$. A densidade compara o quão típico é um valor.

$\sigma = 0{,}2 \rightarrow f(\mu) \approx 1{,}995$

Maior que 1: não é probabilidade.

Gaussian Naive Bayes com números e NumPy



Classe A

$f(6|A) = 0{,}2420$

$0{,}5 \times 0{,}2420 = 0{,}1210$

Classe B

$f(6|B) = 0{,}0540$

$0{,}5 \times 0{,}0540 = 0{,}0270$

Previsão: A · posteriori ≈ 0,8176

```python
import numpy as np
X = np.array([[4.], [4.], [6.], [6.],
              [7.], [7.], [9.], [9.]])   # (8, 1)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])  # 0=A, 1=B
classes = np.unique(y)                    # [0 1]
# treino: média, variância e priori por classe
mu  = np.array([X[y == c].mean(0) for c in classes])
var = np.array([X[y == c].var(0) for c in classes])
prior = np.array([np.mean(y == c) for c in classes])

x_novo = np.array([6.])
# densidade normal de cada característica, por classe
dens = (np.exp(-(x_novo - mu) ** 2 / (2 * var))
        / np.sqrt(2 * np.pi * var))       # (2, 1)
scores = prior * dens.prod(axis=1)  # ≈ [0.1210 0.0270]
print(scores / scores.sum())        # ≈ [0.8176 0.1824]
print(classes[np.argmax(scores)])   # 0 -> classe A
```

Multinomial Naive Bayes: textos viram contagens

Para características que são contagens não negativas, como o número de vezes que cada palavra aparece.

TREINO · 4 MENSAGENS

M1	promoção grátis grátis	Spam
M2	ganhe promoção	Spam
M3	reunião hoje	Normal
M4	agenda reunião	Normal

VOCABULÁRIO · ORDEM FIXA, SÓ DO TREINO

0·agenda 1·ganhe 2·grátis 3·hoje 4·promoção 5·reunião

TOKEN

Cada unidade do texto; aqui, uma palavra. M1 tem 3 tokens.

TOKENIZAÇÃO

Minúsculas, sem pontuação, separar palavras.

VOCABULÁRIO

Palavras diferentes do treino, cada uma com um índice.

BAG OF WORDS

Só as contagens importam; a ordem das palavras é descartada.

Do texto à matriz numérica: Bag of Words

“promoção grátis grátis”	Spam
“ganhe promoção”	Spam
“reunião hoje”	Normal
“agenda reunião”	Normal

→

	agenda	ganhe	grátis	hoje	promoção	reunião	Y
M1	0	0	**2**	0	**1**	0	1
M2	0	**1**	0	0	**1**	0	1
M3	0	0	0	**1**	0	**1**	0
M4	**1**	0	0	0	0	**1**	0

linha = mensagem coluna = palavra célula = contagem `X.shape = (4, 6)`

Contando palavras por classe

PALAVRA	SPAM N	P(W|SPAM)	NORMAL N	P(W|NORMAL)
agenda	0	0/5 = 0,00	1	1/4 = 0,25
ganhe	1	1/5 = 0,20	0	0/4 = 0,00
grátis	2	2/5 = 0,40	0	0/4 = 0,00
hoje	0	0/5 = 0,00	1	1/4 = 0,25
promoção	2	2/5 = 0,40	0	0/4 = 0,00
reunião	0	0/5 = 0,00	2	2/4 = 0,50
Total	5	1	4	1

SEM SUAVIZAÇÃO

$$P(w \mid C) = \frac{N_{C,w}}{N_C}$$

$N_{C,w}$: ocorrências de w na classe C

N_C: total de tokens da classe C

TOKENS ≠ DOCUMENTOS

Tokens: $\frac{2}{5} = 0{,}4$ · Docs: $\frac{1}{2} = 0{,}5$

O Multinomial usa a fração de tokens.

O problema da probabilidade zero

Mensagem nova: “reunião grátis”. Probabilidades sem suavização, contadas no treino.

	priori	×	reunião	×	grátis	=	
Spam	0,5	×	0/5	×	2/5	=	0
Normal	0,5	×	2/4	×	0/4	=	0

CONSEQUÊNCIA

Um único fator zero anula o produto. As duas classes empatam em 0 e o modelo não decide.

RIGIDEZ

Se *P*(grátis | Normal) = 0, nenhuma mensagem com “grátis” pode ser normal, por mais palavras de trabalho que tenha.

Suavização de Laplace: nenhuma palavra com probabilidade zero

$$P(w \mid C) = \frac{N_{C,w} + \alpha}{N_C + \alpha \cdot |V|}$$

$N_{C,w}$	contagem da palavra w na classe C
N_C	total de tokens da classe C
α	contagem fictícia somada a cada palavra; α = 1 é Laplace
$\lvert V \rvert$	tamanho do vocabulário: 6 palavras

SPAM · A = 1

$$P(\text{grátis} \mid \text{Spam}) = \frac{2+1}{5+6} = \frac{3}{11} \approx 0{,}2727$$

NORMAL · A = 1

$$P(\text{grátis} \mid \text{Normal}) = \frac{0+1}{4+6} = \frac{1}{10} = 0{,}1$$

Este α não é a taxa de aprendizado: mesma letra, outro significado.

A tabela de probabilidades antes e depois de Laplace

SPAM · DENOMINADOR 5 + 6 = 11

PALAVRA	FRAÇÃO	P	ANTES · DEPOIS
agenda	0+1 / 11	0,0909	
ganhe	1+1 / 11	0,1818	
grátis	2+1 / 11	0,2727	
hoje	0+1 / 11	0,0909	
promoção	2+1 / 11	0,2727	
reunião	0+1 / 11	0,0909	
Soma	11/11	1	

NORMAL · DENOMINADOR 4 + 6 = 10

PALAVRA	FRAÇÃO	P	ANTES · DEPOIS
agenda	1+1 / 10	0,2000	
ganhe	0+1 / 10	0,1000	
grátis	0+1 / 10	0,1000	
hoje	1+1 / 10	0,2000	
promoção	0+1 / 10	0,1000	
reunião	2+1 / 10	0,3000	
Soma	10/10	1	

A verossimilhança de uma mensagem inteira

$$P(\text{mensagem} \mid C) \propto \prod_{w \in V} P(w \mid C)^{n_w}$$

n_w = vezes que w aparece na mensagem expoente 0 → fator 1 $\propto$: o coeficiente multinomial é igual nas duas classes

EXEMPLO: "GRÁTIS GRÁTIS PROMOÇÃO", A = 1

	agenda	ganhe	grátis	hoje	promoção	reunião
n_w	0	0	2	0	1	0
Spam	$(1/11)^0 = 1$	$(2/11)^0 = 1$	$(3/11)^2$	$(1/11)^0 = 1$	$(3/11)^1$	$(1/11)^0 = 1$
Normal	$(2/10)^0 = 1$	$(1/10)^0 = 1$	$(1/10)^2$	$(2/10)^0 = 1$	$(1/10)^1$	$(3/10)^0 = 1$

Spam $(3/11)^3 = \frac{27}{1331} \approx 0{,}020285$

Normal $(1/10)^3 = \frac{1}{1000} = 0{,}001$

Classificando “grátis promoção” no caderno: os fatores

	PASSO	SPAM	NORMAL
1	Identificar as palavras	grátis, promoção	grátis, promoção
2	Contagens na mensagem	1 e 1	1 e 1
3	*P*(palavra | classe), α = 1	$\frac{3}{11}$ e $\frac{3}{11}$	$\frac{1}{10}$ e $\frac{1}{10}$
4	Priori *P*(classe)	$\frac{2}{4} = 0{,}5$	$\frac{2}{4} = 0{,}5$
5	Multiplicar os fatores	$0{,}5 \times {}^{3}\!/_{11} \times {}^{3}\!/_{11} \approx 0{,}037190$	$0{,}5 \times {}^{1}\!/_{10} \times {}^{1}\!/_{10} = 0{,}005$

Classificando “grátis promoção”: comparação e decisão

6 · COMPARAR PONTUAÇÕES



Soma = 0,042190 ≠ 1: ainda não são probabilidades.

7 · NORMALIZAR

$$P(\text{Spam} \mid \text{msg}) = \frac{0{,}037190}{0{,}042190} \approx 0{,}8815$$

$$P(\text{Normal} \mid \text{msg}) = \frac{0{,}005}{0{,}042190} \approx 0{,}1185$$

PONTUAÇÃO ∝ POSTERIOR

0,037190

Serve para comparar.

POSTERIOR NORMALIZADA

0,8815

Serve para expressar confiança.

CLASSE ESCOLHIDA

Spam

argmax: a resposta final.

Por que usamos logaritmos: o underflow

QUATRO PALAVRAS

$$0{,}01 \times 0{,}02 \times 0{,}001 \times 0{,}003$$

$$= 0{,}000\ 000\ 000\ 6 = 6 \times 10^{-10}$$

CADA PALAVRA COM PROBABILIDADE 0,001

PALAVRAS	PRODUTO	SOMA DOS LOGS
1	0,001	−6,91
10	10^{-30}	−69,08
100	10^{-300}	−690,78
500	`0.0`	−3453,88

Teste no Python: `0.001 ** 500` → `0.0`

UNDERFLOW NUMÉRICO

O float64 não representa positivos menores que ≈ 5×10^{-324}. O produto vira `0.0` sem aviso.

Logaritmo natural: o que precisamos saber

X E LN(X) · NP.LOG(X)

X	LN(X)
1	0
0,5	−0,6931
0,1	−2,3026
0,01	−4,6052
0,001	−6,9078
0	−∞ (indefinido)

Probabilidades entre 0 e 1 têm log negativo.

DEFINIÇÃO

$$\ln x = y \Leftrightarrow e^{y} = x$$

e ≈ 2,7183 · ln 0,5 ≈ −0,6931 porque $e^{-0,6931} \approx 0,5$

PRODUTO VIRA SOMA

$$\ln(a \cdot b) = \ln a + \ln b$$

ln(0,5 × 0,2) = −0,6931 − 1,6094 = −2,3026

POTÊNCIA VIRA MULTIPLICAÇÃO

$$\ln(a^{n}) = n \cdot \ln a$$

$\ln(0,5^{3}) = 3 \times (-0,6931) = -2,0794$

CRESCENTE

$$a > b > 0 \Rightarrow \ln a > \ln b$$

Comparar logs escolhe o mesmo vencedor.

Somas de logaritmos no lugar de produtos

PROPRIEDADE

$$\log(a \times b) = \log a + \log b$$

$\log(0{,}5 \times 0{,}2) = -2{,}3026 = -0{,}6931 + (-1{,}6094)$

LOG-PONTUAÇÃO DA CLASSE C

$$\log P(C) + \Sigma_i \, n_i \cdot \log P(w_i \mid C)$$

n_i = contagem da palavra i na mensagem · log natural

"GRÁTIS PROMOÇÃO" EM LOGS

Spam $-0{,}6931 - 1{,}2993 - 1{,}2993 = -3{,}2917$

Normal $-0{,}6931 - 2{,}3026 - 2{,}3026 = -5{,}2983$

$-3{,}2917 > -5{,}2983 \rightarrow$ Spam. O log é crescente, então a ordem e o argmax se mantêm. Log-probabilidades são negativas: não são probabilidades.

Normalização estável · subtrair o máximo

```
# log-pontuações: [spam, normal]
log_scores = np.array([-3.2917, -5.2983])
m = log_scores.max()            # -3.2917
probs = np.exp(log_scores - m)  # [1. 0.1344]
probs = probs / probs.sum()     # [0.8815 0.1185]
```

Quiz rápido: Laplace, logaritmos e um cálculo no caderno

Q1 Por que uma palavra ausente em uma classe pode causar problemas?

Q2 O que faz a suavização de Laplace?

Q3 Por que utilizamos logaritmos?

Q4 Com $\alpha = 1$, calcule $P(\text{hoje} \mid \text{Normal})$ e $P(\text{hoje} \mid \text{Spam})$.

Q5 Classifique “reunião hoje”: as duas pontuações, a classe e a posterior.

Dados: Spam com 5 tokens, Normal com 4, $|V| = 6$, prioris 0,5. reunião: 0 em spam, 2 em normal. hoje: 0 em spam, 1 em normal.

Gaussian e Multinomial Naive Bayes lado a lado

	GAUSSIAN NB	MULTINOMIAL NB
Tipo de dado	Contínuo: altura, temperatura	Contagens ≥ 0: palavras
Distribuição	Normal por classe e característica	Multinomial sobre o vocabulário
Parâmetros	μ, σ^2 por classe e característica	$P(w \mid C)$ suavizada por classe e palavra
Pontuação	$\log P(C) + \Sigma \log f(x_i \mid C)$	$\log P(C) + \Sigma\, n_i \log P(w_i \mid C)$
Aplicações	Sensores, medições clínicas	Spam, notícias, sentimentos
Limitações	Supõe sino; sofre com variância ≈ 0	Ignora a ordem; exige valores ≥ 0

Nenhuma variante é melhor sempre: depende da representação dos dados. Multinomial NB não é regressão logística multinomial.

Dicionário matemático do Naive Bayes

$P(A)$	**Probabilidade** chance do evento A, de 0 a 1
$\hat{P}$	**Estimativa** probabilidade medida nos dados
$\mid$	**Dado que** separa evento e condição
$\cap$	**Interseção** A e B ao mesmo tempo
C	**Classe** spam ou normal
x, x_i	**Características** vetor e sua posição i
$P(C)$	**Priori** chance da classe antes da evidência
$P(x\mid C)$	**Verossimilhança** chance da evidência na classe
$P(C\mid x)$	**Posteriori** chance da classe após a evidência
$P(x)$	**Evidência** normalizador, igual para todas
$\propto$	**Proporcional a** mesma ordem, sem somar 1
$\prod$	**Produtório** multiplica todos os termos
Σ	**Somatório** soma todos os termos
argmax	**Argumento do máximo** quem tem a maior pontuação
$\hat{y}$	**Previsão** classe escolhida pelo modelo
μ	**Média** centro da distribuição
σ^2	**Variância** espalhamento ao quadrado
σ	**Desvio-padrão** raiz da variância
π	**Pi** ≈ 3,1416
e, $\exp$	**Euler, exponencial** e ≈ 2,7183; exp(z) = e^z
$f(x\mid C)$	**Densidade** não é probabilidade
$N_{C,w}$	**Contagem** tokens w na classe C
N_C	**Total da classe** tokens da classe C
$\lvert V\rvert$	**Vocabulário** nº de palavras diferentes
α	**Suavização** contagem fictícia; não é taxa
n_i	**Contagem na msg** vezes que a palavra i aparece
$\log$	**Logaritmo natural** np.log; vira produto em soma
Ω	**Espaço amostral** todos os resultados possíveis

Da matemática ao código: as 12 etapas

1 Preparar mensagens
textos, y

2 Tokenizar
tokenizar(texto)

3 Vocabulário · só treino
V, |V|

4 Vetorizar contagens
X: (n, |V|)

5 Rótulos das classes
np.unique(y)

6 Prioris
$P(C) = n_C / n$

7 Contar palavras por classe
$N_{C,w}$

8 Laplace
$(N_{C,w} + \alpha) / (N_C + \alpha|V|)$

9 Log-probabilidades
$\log P(w \mid C)$

10 Predição
$\log P(C) + X \cdot \log P$

11 Classe de maior pontuação
argmax

12 Avaliar
acurácia, matriz de confusão

Vetorização · fora do modelo fit() predict() e avaliação

Do texto ao vetor de contagens

```python
import re
import numpy as np

def tokenizar(texto):
    # minúsculas; só sequências de letras (com acentos)
    return re.findall(r"[a-záàâãéêíóôõúç]+", texto.lower())

def construir_vocabulario(textos_treino):
    palavras = set()
    for txt in textos_treino:          # SOMENTE treino
        palavras.update(tokenizar(txt))
    return {p: j for j, p in enumerate(sorted(palavras))}

def vetorizar(textos, vocab):
    X = np.zeros((len(textos), len(vocab)), dtype=int)
    for i, txt in enumerate(textos):
        for t in tokenizar(txt):
            if t in vocab:             # desconhecida: ignorada
                X[i, vocab[t]] += 1
    return X
```

NO MINI DATASET

```
tokenizar("Ganhe já, GRÁTIS!")
['ganhe', 'já', 'grátis']
```

```
vocab
{'agenda': 0, 'ganhe': 1, 'grátis': 2,
 'hoje': 3, 'promoção': 4, 'reunião': 5}
```

```
X = vetorizar(textos, vocab)
X.shape → (4, 6)
```

```
y = np.array([1, 1, 0, 0])
np.unique(y) → [0 1] · 0 = normal, 1 = spam
```

Treino: prioris, contagens, Laplace e logaritmos

```python
alpha = 1.0
classes = np.unique(y)                          # [0 1]

# 6 · prioris P(C): fração de mensagens por classe
priori = np.array([np.mean(y == c) for c in classes])

# 7 · N_{C,w}: soma das linhas de cada classe
contagens = np.array([X[y == c].sum(axis=0)
                      for c in classes])        # (2, 6)

# 8 · Laplace: (N_{C,w} + α) / (N_C + α|V|)
totais = contagens.sum(axis=1, keepdims=True)   # (2, 1)
prob = (contagens + alpha) / (totais + alpha * X.shape[1])
assert np.allclose(prob.sum(axis=1), 1)

# 9 · log-probabilidades
log_priori = np.log(priori)                     # (2,)
log_prob = np.log(prob)                         # (2, 6)
```

VALORES · LINHA 0 = NORMAL, 1 = SPAM

priori

```
[0.5 0.5]
```

contagens

```
[[1 0 0 1 0 2]
 [0 1 2 0 2 0]]
```

totais

```
[[4]
 [5]]
```

prob[1] · spam

```
[0.0909 0.1818 0.2727
 0.0909 0.2727 0.0909]
```

O produto X @ log_prob.T, passo a passo

PALAVRA	X	LOG P(W|N)	X · LOG	LOG P(W|S)	X · LOG
agenda	0	−1,6094	0	−2,3979	0
ganhe	0	−2,3026	0	−1,7047	0
grátis	1	−2,3026	−2,3026	−1,2993	−1,2993
hoje	0	−1,6094	0	−2,3979	0
promoção	1	−2,3026	−2,3026	−1,2993	−1,2993
reunião	0	−1,2040	0	−2,3979	0
Σ			−4,6052		−2,5986
+ log priori			−0,6931		−0,6931
Pontuação			−5,2983		−3,2917

FORMATOS

```
x              (1, 6)
log_prob.T (6, 2)
x @ log_prob.T (1, 2)
```

O QUE @ CALCULA

$$\Sigma_i \, n_i \cdot \log P(w_i \mid C)$$

Um produto escalar por classe. A dimensão interna (6) exige o mesmo vocabulário.

Predição: log-pontuações, argmax e probabilidades

```python
def log_pontuacoes(X, log_priori, log_prob):
    # (n, |V|) @ (|V|, 2) -> (n, 2), + log P(C)
    return X @ log_prob.T + log_priori

def prever(X, classes, log_priori, log_prob):
    S = log_pontuacoes(X, log_priori, log_prob)
    return classes[np.argmax(S, axis=1)]

def prever_proba(X, log_priori, log_prob):
    S = log_pontuacoes(X, log_priori, log_prob)
    S = S - S.max(axis=1, keepdims=True)  # estabilidade
    E = np.exp(S)
    return E / E.sum(axis=1, keepdims=True)

def acuracia(y_true, y_pred):
    return np.mean(y_true == y_pred)
```

"GRÁTIS PROMOÇÃO"

```
x = vetorizar(["grátis promoção"], vocab)
[[0 0 1 0 1 0]]
```

```
log_pontuacoes(x, ...) · [normal, spam]
[[-5.2983 -3.2917]]
```

```
prever(x, ...)
[1] → spam
```

```
prever_proba(x, ...)
[[0.1185 0.8815]]
```

Organizando tudo na classe NaiveBayesDoZero

```python
class NaiveBayesDoZero:
    def __init__(self, alpha=1.0):
        self.alpha = alpha            # suavização de Laplace

    def fit(self, X, y):
        # etapas 5-9: aprende classes_, log_priori_,
        # contagens_ e log_prob_
        ...
        return self

    def predict(self, X):
        # etapas 10-11: argmax das log-pontuações
        ...

    def predict_proba(self, X):
        # subtrair o máximo, exp, normalizar
        ...

    def score(self, X, y):
        return np.mean(self.predict(X) == y)
```

CONTRATO DE X

Contagens ≥ 0, colunas na ordem do vocabulário de treino. A classe não conhece texto.

Uso

```python
vocab = construir_vocabulario(txt_treino)
X_tr = vetorizar(txt_treino, vocab)
X_te = vetorizar(txt_teste, vocab)
nb = NaiveBayesDoZero(alpha=1.0)
nb.fit(X_tr, y_tr)
print(nb.score(X_te, y_te))
```

Exercício guiado: o dataset e a divisão treino/teste

32	24	8	102
mensagens	treino · 12/12	teste · 4/4	palavras no vocab

Promoção imperdível ganhe desconto hoje	spam
Clique no link e receba seu prêmio	spam
Reunião de equipe amanhã às dez horas	normal
Segue o relatório da reunião de ontem	normal

```
teste = [i for i in range(len(textos)) if i % 4 == 3]
treino = [i for i in range(len(textos)) if i % 4 != 3]
```

ROTEIRO DO NOTEBOOK

1	Criar o dataset	2	Visualizar mensagens
3	Identificar classes	4	Separar treino e teste
5	Vocabulário só do treino	6	Textos em contagens
7	Prioris das classes	8	Probabilidades das palavras
9	Aplicar Laplace	10	Usar logaritmos
11	Treinar	12	Gerar previsões
13	Acurácia	14	Matriz de confusão
15	Testar mensagens novas	16	Palavras mais influentes

Resultados no conjunto de teste

MENSAGEM DE TESTE	REAL	PREVISTO	P(SPAM)
Oferta exclusiva grátis para você	spam	spam	0,9686
Receba seu bônus agora clique aqui	spam	spam	0,9879
Empréstimo aprovado receba dinheiro agora	spam	spam	0,8905
Oferta imperdível receba agora seu presente	spam	spam	0,9850
Vamos almoçar juntos na sexta	normal	normal	0,2017
Obrigado pela ajuda no projeto	normal	normal	0,4312
Parabéns você ganhou o prêmio de melhor projeto	normal	spam	0,7570
Bom dia segue a agenda atualizada	normal	normal	0,0350

MATRIZ DE CONFUSÃO · LINHAS = REAL

	prev. normal	prev. spam
normal	3 VN	1 FP
spam	0 FN	4 VP

Acurácia
7/8 = 0,875

Precisão spam
4/5 = 0,8

Recall spam
4/4 = 1,0

No treino: 24/24. Avaliar só no treino seria otimista.

Como o modelo classifica “Parabéns! Você ganhou uma promoção!”

TOKEN	N · S	LOG P(W|NORMAL)	LOG P(W|SPAM)
parabéns	0 · 2	−5,2204	−4,1163
você	1 · 2	−4,5272	−4,1163
ganhou	0 · 1	−5,2204	−4,5218
uma	—	fora do vocabulário	ignorada
promoção	0 · 4	−5,2204	−3,6055
log priori	12 · 12 msgs	−0,6931	−0,6931
Soma		−20,8814	−17,0531

LAPLACE COM O VOCABULÁRIO REAL

$$P(\text{promoção} \mid \text{Spam}) = \frac{4 + 1}{82 + 102} \approx 0{,}02717$$

$$P(\text{promoção} \mid \text{Normal}) = \frac{0 + 1}{83 + 102} \approx 0{,}00541$$

DECISÃO

−17,0531 > −20,8814 → spam

$P(\text{spam} \mid \text{msg}) \approx 0{,}9787$

As palavras que mais pesam em cada classe

Diferença log $P(w \mid \text{Spam})$ − log $P(w \mid \text{Normal})$, calculada no treino. Contagens: spam · normal. Empates em ordem alfabética.



+1,797 → “ganhe” é $e^{1,797} \approx 6$ vezes mais provável em spam. “a”, “o” e “da” são artefatos de um dataset pequeno.

Erros comuns (1 de 3)

#	ERRO	POR QUE ESTÁ ERRADO	CONSEQUÊNCIA	CORREÇÃO
1	Confundir $P(A\|B)$ com $P(B\|A)$	A condição define o universo de contagem	Ler 0,75 = P(Grátis|Spam) como “75% de spam”	Ler a barra como “dado que”; conferir o denominador
2	Confundir priori e posteriori	Priori vem antes da evidência; posteriori, depois	Decidir só pela proporção das classes	Priori = $P(C)$; posteriori = $P(C\|x)$
3	Independência condicional tratada como absoluta	A hipótese vale dentro de cada classe	Explicação errada do modelo e de seus limites	$P(x_1, x_2\|C) = P(x_1\|C)\,P(x_2\|C)$
4	Trocar Gaussian por Multinomial	Cada variante supõe uma distribuição	Probabilidades sem sentido para o dado	Contínuo → Gaussian; contagens → Multinomial
5	Frequências incorretas	Dividir por nº de mensagens ou por $\|V\|$	$P(w\|C)$ que não somam 1	Dividir pelo total de tokens da classe

Erros comuns (2 de 3)

#	ERRO	POR QUE ESTÁ ERRADO	CONSEQUÊNCIA	CORREÇÃO
6	Contar documentos em vez de tokens	O Multinomial modela ocorrências	0,5 em vez de 0,4 para “grátis”	Somar as linhas de X por classe
7	Esquecer a suavização de Laplace	Palavra ausente gera probabilidade 0	Produto 0, log(0) = –inf, empates	Somar α a todas as contagens
8	Denominador de Laplace errado	Somar α só no numerador desequilibra	Probabilidades somam mais que 1	N_C + α|V|; assert soma = 1
9	Multiplicar muitas probabilidades	float64 não representa < 5 × 10^{-324}	Pontuações 0.0: underflow	Somar log-probabilidades
10	Log-probabilidade lida como probabilidade	log de valores ≤ 1 é ≤ 0	“−3,29 de chance”	exp com normalização estável

Erros comuns (3 de 3)

#	ERRO	POR QUE ESTÁ ERRADO	CONSEQUÊNCIA	CORREÇÃO
11	Vocabulário com o teste	Informação do teste vaza para o treino	Avaliação otimista	Vocabulário só com textos de treino
12	Características negativas	Multinomial exige contagens ≥ 0	Modelo matematicamente inválido	assert (X >= 0).all()
13	Ordem do vocabulário diferente	Coluna j passa a ser outra palavra	Probabilidades aplicadas à palavra errada	Reutilizar o mesmo dicionário vocab
14	Palavra desconhecida sem regra	Ela não tem coluna	KeyError ou previsões inconsistentes	Regra explícita: ignorar e documentar
15	Avaliar só no treino	O modelo já viu esses dados	Treino 24/24 esconde o erro do teste	Medir acurácia e matriz no teste

Desafio: construa um filtro de spam com Naive Bayes do zero

DADOS

1. Criar ou carregar o dataset
2. Separar treino e teste
3. Vocabulário só com o treino
4. Textos em vetores de contagem

MODELO

5. Prioris das classes
6. Frequências das palavras
7. Suavização de Laplace
8. Log-pontuações
9. fit()
10. predict()
11. predict_proba()

AVALIAÇÃO

12. Acurácia e matriz de confusão
13. Testar mensagens novas
14. Explicar duas previsões

VOCÊ RECEBE

Notebook com enunciado, esqueleto da classe, TODOs e testes automáticos.

DATASET NOVO

48 mensagens em português, 24 spam e 24 normais, diferentes das do exercício guiado.

EXTRAS OPCIONAIS

Remover stopwords, variar α e comparar com o MultinomialNB.

Critérios de aceitação e entrega

1. O programa executa sem erros
2. As probabilidades estão corretas
3. A suavização está correta
4. Não há vazamento entre treino e teste
5. Previsões = maiores pontuações
6. predict_proba soma 1 por linha
7. Funciona com mensagens novas
8. Métricas coerentes com as previsões
9. O aluno explica cada etapa

TESTE RÁPIDO DO NÚCLEO

No mini dataset, “grátis promoção” deve dar

$$P(\text{spam}) \approx 0{,}8815$$

ENTREGA

Notebook do desafio preenchido e executado, com um parágrafo sobre os erros do modelo.

Leitura complementar: documentação oficial

NumPy · documentação oficial	numpy.org/doc/stable
numpy.log	numpy.org · reference · numpy.log
scikit-learn · Naive Bayes	scikit-learn.org/stable/modules/naive_bayes.html
GaussianNB	scikit-learn.org · sklearn.naive_bayes.GaussianNB
MultinomialNB	scikit-learn.org · sklearn.naive_bayes.MultinomialNB
Extração de características de texto	scikit-learn.org · feature_extraction · texto
Python · módulo re	docs.python.org/3/library/re.html